Tarefa 14: Programação em Memória Distribuída

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

Em programação paralela com memória distribuída, o custo de comunicar uma mensagem entre dois processos depende de duas características do sistema: a *latência* (α), tempo fixo para iniciar qualquer comunicação, e a *largura de banda* (β), taxa sustentada de transferência de dados. Um modelo simples e muito usado para o tempo de uma mensagem unidirecional de n bytes é o modelo de Hockney:

$$T(n) = \alpha + \frac{n}{\beta}. \tag{1}$$

Este trabalho mede $T(n)$ com um teste de *ping-pong* em MPI, executado no supercomputador NPAD (UFRN), e identifica os regimes em que a latência domina (mensagens pequenas) e em que a largura de banda domina (mensagens grandes).

2 Metodologia

O programa (Código 1, no Apêndice) usa exatamente dois processos. O processo 0 envia uma mensagem de n bytes ao processo 1 com `MPI_Send`, e o processo 1 a devolve imediatamente com `MPI_Send`, enquanto o processo 0 a recebe com `MPI_Recv`. O tempo de várias trocas consecutivas é medido com `MPI_Wtime`, após um `MPI_Barrier`.

Detalhes do experimento:

- tamanhos de 8 bytes ($\approx 10^1$) a 64 MiB ($\approx 7 \times 10^7$ bytes), em potências de 2 (2^3 a 2^{26});
- número de repetições adaptativo por tamanho (de 20 a 20 000 idas e voltas), para que cada medição tenha duração suficiente;
- 20 trocas de aquecimento não medidas e 5 tentativas por tamanho, sendo registrada a de menor tempo (para reduzir ruído);
- o buffer é inicializado antes da medição (evita *page faults*) e o conteúdo devolvido é verificado ao final de cada tentativa;
- o tempo unidirecional é $T(n) = t_{\text{total}}/(2 \cdot \text{reps})$ e a banda efetiva é $n/T(n)$.

O experimento foi executado em dois cenários no NPAD, com o script bash (2):

1. **intra-nó**, com os dois processos no mesmo nó (comunicação via memória compartilhada)

2. **inter-nó**, com um processo em cada nó (comunicação pela rede de interconexão).

Os resultados foram processados com `plot_pingpong.py`, que também ajusta o modelo de Hockney por mínimos quadrados com pesos $1/T$ (erro relativo) e calcula $n_{1/2} = \alpha\beta$, o tamanho a partir do qual o termo de banda supera o de latência.

3 Resultados

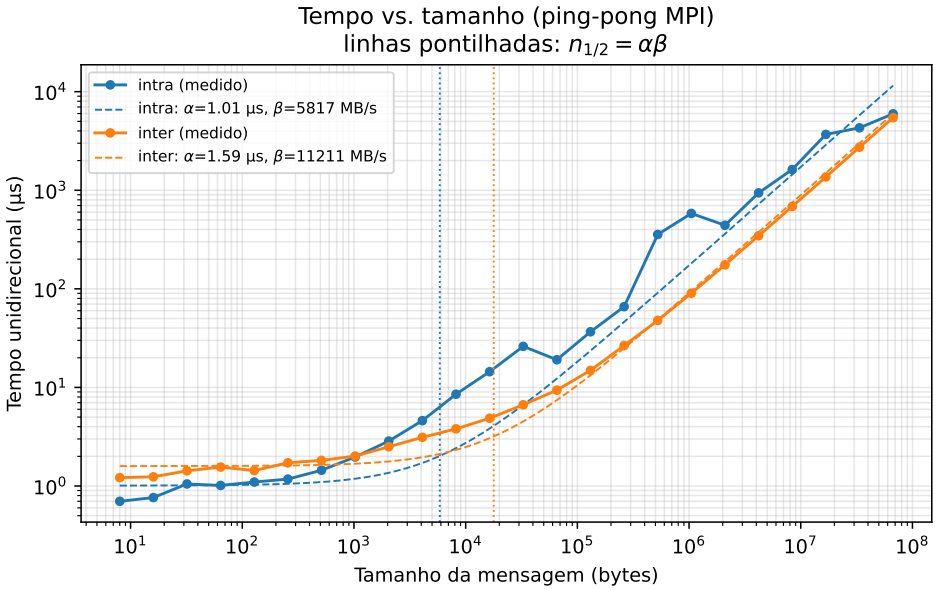


Figura 1: Tempo unidirecional em função do tamanho da mensagem (escala log-log), com ajuste do modelo de Hockney.

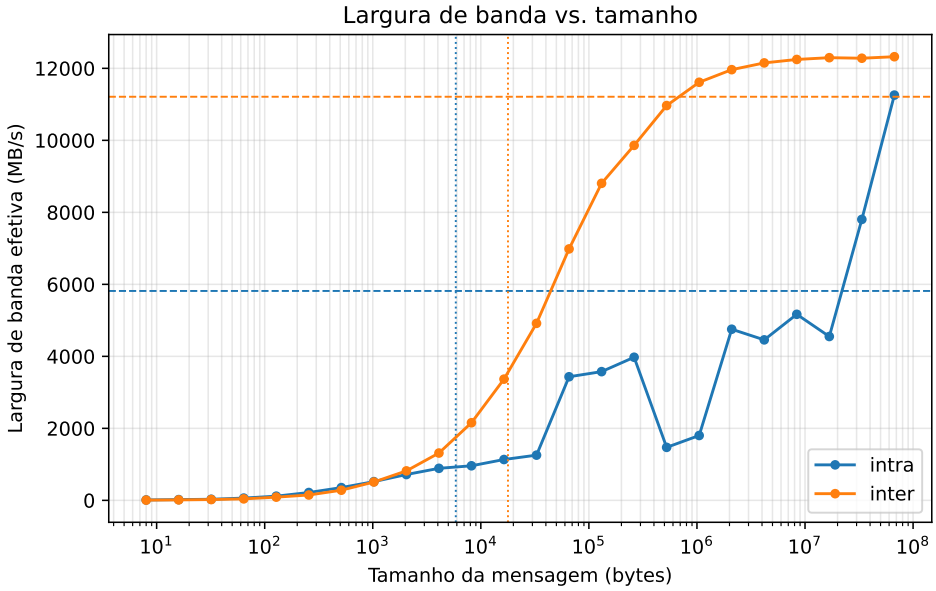


Figura 2: Largura de banda efetiva em função do tamanho da mensagem.

Cenário	α (μs)	β (MB/s)	$n_{1/2}$ (bytes)
Intra-nó	1.008	5816.5	5865
Inter-nó	1.591	11210.8	17835

Tabela 1: Parâmetros do modelo de Hockney ajustados.

4 Análise

4.1 Como ler os gráficos

No eixo horizontal, cada marca é 10 vezes maior que a anterior ($10^1, 10^2, \ldots, 10^8$ bytes). No gráfico de tempo, quanto mais alta a curva, mais lenta a comunicação; no de banda, quanto mais alta, melhor. Os valores abaixo são aproximados, lidos dos gráficos (exceto os da Tabela 1).

4.2 Os dois regimes (inter-nó)

Regime	Tamanho da mensagem	O que se vê
Latência domina	até $\approx 10^3$ bytes	o tempo quase não muda ($\approx$ 1,2 a 2 μs); a banda é menor que 5% do pico
Transição	$\approx 10^3$ a $\approx 10^5$ bytes	latência e banda pesam parecido
Banda domina	a partir de $\approx 10^5$ bytes	o tempo cresce na mesma proporção do tamanho; a banda sobe até $\approx$ 12,3 GB/s ($> 90\%$ do pico em $\approx 10^6$ bytes)

Tabela 2: Resumo dos regimes no cenário inter-nó.

Simplificando:

- **Mensagens pequenas** (até $\approx 10^3$ bytes): mandar 10 bytes ou 1000 bytes leva quase o mesmo tempo. O que pesa é o custo fixo de iniciar a comunicação, a **latência**.
- **Mensagens grandes** ($\approx 10^5$ bytes ou mais): o tempo passa a ser proporcional ao tamanho. O que pesa é transmitir os dados, a **largura de banda**. Em $\approx 7 \times 10^7$ bytes, a latência é menos de 0,1% do tempo.
- **Ponto de equilíbrio:** $n_{1/2} = \alpha\beta \approx 1{,}8 \times 10^4$ bytes (Tabela 1), no meio da faixa de transição.

	Inter-nó		Intra-nó	
Tamanho (bytes)	Tempo (μs)	Banda (MB/s)	Tempo (μs)	Banda (MB/s)
8 ($\approx 10^1$)	$\approx$ 1,2	$\approx$ 7	$\approx$ 0,7	$\approx$ 11
1 024 ($\approx 10^3$)	$\approx$ 2	$\approx$ 500	$\approx$ 2	$\approx$ 500
131 072 ($\approx 10^5$)	$\approx$ 15	$\approx$ 8 800	$\approx$ 35	$\approx$ 3 600
1 048 576 ($\approx 10^6$)	$\approx$ 90	$\approx$ 11 600	$\approx$ 580	$\approx$ 1 800
8 388 608 ($\approx 10^7$)	$\approx$ 700	$\approx$ 12 200	$\approx$ 1 600	$\approx$ 5 200

Tabela 3: Pontos de referência (tempo unidirecional e banda efetiva), aproximados e lidos dos gráficos.

4.3 Intra-nó vs. inter-nó

- **Mensagens pequenas** ($\approx 10^1$ bytes): o intra-nó é mais rápido ($\approx$ 0,7 μs contra $\approx$ 1,2 μs), pois não passa pela rede.
- **A partir de $\approx 10^5$ bytes:** o inter-nó é melhor. O intra-nó dá um salto entre 10^5 e 10^6 bytes ($\approx 5 \times 10^5$), onde o inter-nó é cerca de 7 vezes mais rápido.
- **Modelo de Hockney:** descreve bem o inter-nó e mal o intra-nó, que é irregular; β e $n_{1/2}$ do intra-nó são só aproximados.
- **Causas possíveis** (hipóteses): cópia extra pela memória, efeitos de cache e posicionamento dos processos nos soquetes.

5 Conclusão

- A **latência domina** até $\approx 10^3$ bytes e a **largura de banda domina** a partir de $\approx 10^5$ bytes. O equilíbrio fica em $n_{1/2} \approx 1{,}8 \times 10^4$ bytes (inter-nó).
- Números principais (inter-nó): $\alpha \approx$ 1,6 μs e $\beta \approx$ 11,2 GB/s (patamar medido de $\approx$ 12,3 GB/s).
- **Na prática:** Enviar 1024 mensagens de 8 bytes custa $\approx$ 1,2 ms, contra $\approx$ 3,7 μs para uma única mensagem com os mesmos $\approx 8 \times 10^3$ bytes (mais de 300 vezes menos).
- O intra-nó tem latência menor, mas desempenho irregular para mensagens médias e grandes.

A Código-Fonte

```c
/** TAREFA 14 - PingPong MPI
 *
 * Compilar: mpicc -O2 -Wall -o tarefa14 tarefa14.c
 *
 * Executar: mpirun -np 2 ./tarefa14 [saida.csv]
 */

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MIN_EXP    3       /* 2^3 = 8 bytes*/
#define MAX_EXP    26      /* 2^26 = 64 MiB*/
#define WARMUP     20
#define TRIALS     5
#define MIN_REPS   20
#define MAX_REPS   20000
#define TOTAL_BYTES (256UL * 1024 * 1024)
#define TAG        0

static void pingpong(char *buf, int n, int reps, int rank)
{
    for (int i = 0; i < reps; i++) {
        if (rank == 0) {
            MPI_Send(buf, n, MPI_CHAR, 1, TAG, MPI_COMM_WORLD);
            MPI_Recv(buf, n, MPI_CHAR, 1, TAG, MPI_COMM_WORLD,
                    MPI_STATUS_IGNORE);
        } else {
            MPI_Recv(buf, n, MPI_CHAR, 0, TAG, MPI_COMM_WORLD,
                    MPI_STATUS_IGNORE);
            MPI_Send(buf, n, MPI_CHAR, 0, TAG, MPI_COMM_WORLD);
        }
    }
}


int main(int argc, char **argv)
{
    int rank, size;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2) {
        if (rank == 0) {
            fprintf(stderr, "Erro: execute com 2 processos "
                            "(mpirun -np 2).\n");
            MPI_Finalize();
            return EXIT_FAILURE;
        }
    }

    size_t max_bytes = (size_t)1 << MAX_EXP;
    char *buf = malloc(max_bytes);
```

```
57
58      if (!buf) {
59          fprintf(stderr, "Rank %d: falha ao alocar %zu bytes.\n", rank, max_bytes);
60          MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
61      }
62      /* Toca as páginas */
63      memset(buf, rank + 1, max_bytes);
64
65      FILE *out = stdout;
66      if (rank == 0 && argc > 1) {
67          out = fopen(argv[1], "w");
68          if (!out) {
69              perror("fopen");
70              MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
71          }
72      }
73
74      if (rank == 0) {
75          fprintf(out, "tamanho_bytes,repeticoes,tempo_total_s,rtt_s,
                tempo_unidirecional_s,banda_MBps\n");
76      }
77
78      for (int e = MIN_EXP; e <= MAX_EXP; e++) {
79          int n = 1 << e;
80          int reps = (int)(TOTAL_BYTES / (size_t)n);
81
82          if (reps > MAX_REPS) reps = MAX_REPS;
83          if (reps < MIN_REPS) reps = MIN_REPS;
84
85          /* AQUECIMENTO */
86          pingpong(buf, n, WARMUP, rank);
87
88          double best = 1e30;
89
90          for (int t = 0; t < TRIALS; t++) {
91              if (rank == 0) {
92                  for (int i = 0; i < n; i++) buf[i] = (char)((i + e) & 0x7F);
93              }
94
95              MPI_Barrier(MPI_COMM_WORLD);
96              double t0 = MPI_Wtime();
97              pingpong(buf, n, reps, rank);
98              double t1 = MPI_Wtime();
99
100             if (rank == 0) {
101                 double dt = t1 - t0;
102                 if (dt < best) best = dt;
103
104                 for (int i = 0; i < n; i++) {
105                     if (buf[i] != (char)((i + e) & 0x7F)) {
106                         fprintf(stderr, "Erro: eco corrompido (n=%d)\n", n);
107                         MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
108                     }
109                 }
110             }
111         }
112
113         if (rank == 0) {
```

```c
            double rtt = best / reps;           /* round trip */
            double oneway = rtt / 2.0;          /* one way */
            double bw = (double)n / oneway / 1e6; /* MB/s */
            fprintf(out, "%d,%d,%.9f,%.9e,%.9e,%.3f\n",
                    n, reps, best, rtt, oneway, bw);
            fflush(out);
        }
    }

    if (rank == 0 && out != stdout) fclose(out);
    free(buf);
    MPI_Finalize();
    return EXIT_SUCCESS;
}
```

Código 1: Ping-pong MPI (`tarefa14.c`).

```bash
#!/bin/bash
#SBATCH --job-name=pingpong
#SBATCH --time=0-1:0
#SBATCH --output=pingpong_%j.out
#SBATCH --error=pingpong_%j.err

#SBATCH --partition=amd-512
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=2
#SBATCH --exclusive

# Pegamos os dois primeiros nos alocados
NODES=($(scontrol show hostnames "$SLURM_JOB_NODELIST"))
N0=${NODES[0]}
N1=${NODES[1]}

echo "Nos: $N0 $N1"

# --- Caso 1: INTRA-NO (2 processos no mesmo no, memoria compartilhada) ---
srun --nodes=1 --ntasks=2 --nodelist="$N0" ./tarefa14 resultados_intra.csv

# --- Caso 2: INTER-NO (1 processo por no, comunicacao pela rede) ---
srun --nodes=2 --ntasks=2 --ntasks-per-node=1 --nodelist="$N0,$N1" \
    ./tarefa14 resultados_inter.csv

echo "Concluido."
```

Código 2: Script bash (`tarefa14.sh`)